

NewBoxofficeProject

Manuale utente e guida ai comandi

Pipeline box office italiano: ComingSoon → TMDB → PostgreSQL → sito vetrina
Aggiornato al 25 settembre 2026

Indice

1. Cos'è il progetto
2. Prerequisiti e installazione
3. Configurazione e credenziali
4. Comandi principali (riepilogo)
5. Flussi di lavoro tipici
6. Esecuzione pianificata e notifiche
7. Test
8. Risoluzione dei problemi
9. Limiti da conoscere

1. Cos'è il progetto

NewBoxofficeProject raccoglie ogni settimana la classifica del box office italiano da **ComingSoon.it**, collega ogni film alla scheda **TMDB** (`movie_id`) e salva tutto in **PostgreSQL**. I KPI vengono poi esportati verso il sito vetrina (repository separato `boxoffice-site`).

ComingSoon → weekly_run → PostgreSQL → resolve_matches (TMDB) → health_check → export KPI
→ sito

Tutti i comandi si lanciano da **PowerShell** nella cartella radice del progetto, con il virtualenv attivo (o usando `.venv\Scripts\python.exe`).

2. Prerequisiti e installazione

- Windows con PowerShell, Python 3.14 e PostgreSQL 18 in locale.
- Un account TMDB (token di lettura o chiave API).

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Prima configurazione (una tantum)

```
python scripts/manage_secrets.py import-env --strip      # sposta gli eventuali segreti di .env
nel deposito
python scripts/setup_db_roles.py --database boxoffice    # crea i ruoli, i privilegi e le
password
# poi imposta DB_USER=boxoffice_app in .env
python scripts/migrate.py                                # crea/aggiorna lo schema
python scripts/manage_secrets.py set tmdb_read_token    # chiede il token senza mostrarlo
```

3. Configurazione e credenziali

Copia `.env.example` in `.env` . Il file contiene **solo valori non segreti**.

Variabile	Uso
<code>DB_HOST</code> , <code>DB_NAME</code> , <code>DB_USER</code>	Obbligatorie (es. localhost, boxoffice, boxoffice_app)
<code>DB_PORT</code> , <code>LOG_LEVEL</code> , <code>REQUEST_TIMEOUT</code>	Opzionali, hanno un valore predefinito
<code>DB_OWNER_USER</code> , <code>DB_RO_USER</code>	Nomi dei ruoli owner (migrazioni, backup) e sola lettura (controlli)

Password e chiavi API stanno nella **Gestione credenziali di Windows** (servizio `NewBoxofficeProject`). Le variabili d'ambiente `DB_PASSWORD` , `TMDB_API_KEY` , `TMDB_READ_TOKEN` hanno la precedenza, e servono alla CI.

Ruoli PostgreSQL

Ruolo	Chi lo usa	Permessi
<code>boxoffice_owner</code>	migrate, backup	Proprietario, unico con DDL
<code>boxoffice_app</code>	pipeline, resolver, Wayback	SELECT / INSERT / UPDATE
<code>boxoffice_ro</code>	health_check, verify_db	Solo SELECT
<code>boxoffice_test</code>	test di integrazione	Solo database temporanei

Database di sviluppo

Per sperimentare senza toccare i dati reali usa `boxoffice_dev` (va creato a mano con `CREATE DATABASE` e inizializzato con `migrate.py`):

```
$env:DB_NAME = "boxoffice_dev"; python scripts/weekly_run.py
```

4. Comandi principali (riepilogo)

Esecuzione della pipeline

Comando	Cosa fa
<code>python main_v3.py</code>	Pipeline completa: carica la classifica corrente, poi abbina i film a TMDB.
<code>python scripts/weekly_run.py</code>	Solo caricamento della classifica ComingSoon corrente. Opzioni: <code>--source-name</code> , <code>--historical-start-date</code> , <code>--weeks</code> .
<code>python scripts/resolve_matches.py</code>	Abbina a TMDB i film senza <code>movie_id</code> .
<code>python scripts/resolve_matches.py --dry-run</code>	Valuta gli abbinamenti senza scrivere.
<code>python scripts/resolve_matches.py --review</code>	Elenca i match incerti con i candidati.
<code>python scripts/resolve_matches.py --set ID --tmdb-id N</code>	Conferma a mano un match (ID di <code>source_movies</code> preso da <code>--review</code>).
<code>python scripts/resolve_matches.py --no-match ID</code>	Segna il film come non presente su TMDB.
<code>python scripts/resolve_matches.py --retry</code>	Riprova anche i casi in revisione o senza match.
<code>python scripts/bootstrap.py</code>	Facoltativo, fuori pipeline: carica i film TMDB più redditizi.

Controlli

Comando	Cosa fa
<code>python scripts/health_check.py</code>	Controlli di salute; esce con codice 1 se c'è un FAIL. Opzioni: <code>-strict</code> (anche i WARN sono errori), <code>--max-age-days</code> , <code>--gap-weeks</code> , <code>--territory</code> .
<code>python scripts/verify_db.py</code>	Conteggi e copertura dei <code>movie_id</code> (ruolo sola lettura).
<code>python test_project.py</code>	Report di verifica di configurazione e ambiente.

Database e dati

Comando	Cosa fa
<code>python scripts/migrate.py</code>	Applica le migrazioni mancanti e riapplica i privilegi.
<code>python scripts/migrate.py --status</code>	Mostra lo stato delle migrazioni senza modificare nulla.
<code>python scripts/backup_db.py --label prima_di_x</code>	Backup con <code>pg_dump</code> in <code>backups/</code> . Retention automatica: 30 giorni, minimo 3 dump (<code>--keep-days</code> , <code>--keep-min</code>).
<code>python scripts/export_site_data.py</code>	Esporta i KPI in JSON (<code>--out</code> per il percorso). Non scrive nulla se l'health check è in FAIL.
<code>python scripts/recover_from_wayback.py -dry-run</code>	Mostra quali settimane mancanti si possono recuperare dall'Internet Archive.
<code>python scripts/recover_from_wayback.py</code>	Scriva le settimane recuperabili (poi lanciare <code>resolve_matches.py</code>).

Credenziali e sicurezza

Comando	Cosa fa
<code>python scripts/manage_secrets.py list</code>	Elenca i segreti presenti (mai i valori).
<code>python scripts/manage_secrets.py set NOME</code>	Salva un segreto; il valore è chiesto senza mostrarlo (<code>--stdin</code> per leggerlo da input).
<code>python scripts/manage_secrets.py get NOME --show</code>	Mostra un segreto.
<code>python scripts/manage_secrets.py delete NOME</code>	Elimina un segreto.
<code>python scripts/setup_db_roles.py --verify-only</code>	Verifica cosa i ruoli possono e non possono fare, senza modificare nulla.
<code>python scripts/setup_db_roles.py --rotate</code>	Genera nuove password per tutti i ruoli.
<code>.\scripts\harden_permissions.ps1</code>	Limita i permessi di <code>.env</code> , <code>backups/</code> , <code>logs/</code> al solo utente corrente.

Pianificazione

Comando	Cosa fa
<code>.\scripts\run_weekly.ps1</code>	Esecuzione non presidiata: pipeline, health check, notifica dei guasti, export KPI verso il sito. Scrive <code>logs\weekly_<data>.log</code> .
<code>.\scripts\register_weekly_task.ps1</code>	Registra l'attività di Windows (lunedì e venerdì alle 12:00). <code>-Time 09:30</code> cambia l'orario, <code>-Remove</code> la elimina.
<code>Start-ScheduledTask -TaskName "NewBoxOffice-WeeklyRun"</code>	Avvio manuale dell'attività pianificata.

5. Flussi di lavoro tipici

Routine settimanale (manuale)

```
python main_v3.py
python scripts/health_check.py
python scripts/verify_db.py
```

Revisione dei match dubbi

Un film è abbinato in automatico se il punteggio è ≥ 0.85 con almeno 0.10 di margine sul secondo candidato. Tra 0.55 e la soglia va in revisione; sotto è "senza match".

```
python scripts/resolve_matches.py --review          # trova l'ID del film e i candidati
python scripts/resolve_matches.py --set 2 --tmdb-id 920 # conferma il candidato giusto
python scripts/resolve_matches.py --no-match 28      # oppure: non esiste su TMDB
```

Recupero di una settimana persa

```
python scripts/recover_from_wayback.py --dry-run
python scripts/recover_from_wayback.py
python scripts/resolve_matches.py
```

Prima di un'operazione rischiosa

```
python scripts/backup_db.py --label prima_di_x
python scripts/migrate.py --status
```

Interrogare i dati

La vista `v_weekly_enriched` unisce classifica, film TMDB e stato del match ed è il punto di accesso consigliato per report e query.

```
SELECT rank, external_movie_title, tmdb_title, weekly_gross, gross_per_screen
FROM v_weekly_enriched
WHERE week_start = (SELECT max(week_start) FROM weekly_box_office)
ORDER BY rank;
```

6. Esecuzione pianificata e notifiche

ComingSoon **non ha archivio**: una settimana non caricata non si recupera facilmente, quindi la pipeline va eseguita ogni settimana. L'attività pianificata parte il lunedì e il venerdì alle 12:00 (il venerdì è una rete di sicurezza se il PC era spento). L'upsert è idempotente: rieseguire non crea duplicati.

- Gira come l'utente corrente **solo con la sessione aperta**; se il PC era spento parte appena possibile.
- I log sono in `logs/` e si cancellano dopo 90 giorni.
- Se la pipeline fallisce o l'health check trova un FAIL: notifica desktop di Windows e, facoltativamente, webhook (compatibile Slack/Discord) impostabile con `python scripts/manage_secrets.py set notify_webhook_url`.
- Il codice di uscita compare come "Ultimo risultato" nell'Utilità di pianificazione.

7. Test

```
python -m unittest discover --start-directory tests --verbose

# test di integrazione con PostgreSQL (database temporaneo)
$env:RUN_DB_TESTS = "1"; python -m unittest tests.test_db_integration -v
```

I test standard non richiedono PostgreSQL, TMDb né `.env`. La CI su GitHub esegue la suite con Python 3.14, un servizio PostgreSQL 18, `bandit` e `pip-audit`.

8. Risoluzione dei problemi

Sintomo	Cosa fare
Errore "migrazioni da applicare"	Lancia <code>python scripts/migrate.py</code> . L'applicazione non esegue DDL.
Health check in FAIL	Leggi il messaggio: pipeline ferma o dati incoerenti. Controlla l'ultimo log in <code>logs/</code> . I soli WARN (es. settimane storiche già perse) non sono guasti.
Settimana mancante	Prova <code>recover_from_wayback.py</code> ; Wayback ha pochi snapshot, non tutte le settimane sono recuperabili.
<code>resolve_matches.py</code> esce con 1	La connessione al database è caduta a metà elenco: rilancia il comando, riprende dai film rimasti.
Errore sul deposito credenziali	Verifica con <code>manage_secrets.py list</code> che il segreto esista; controlla di essere nella sessione utente corretta.

Nessuna notifica ricevuta	La notifica desktop funziona solo a sessione aperta; per il webhook controlla <code>notify_webhook_url</code> .
Un film ha un <code>[errore]</code> nel resolver	È un guasto TMDb o dati di quel solo film: gli altri proseguono. Riprova con <code>--retry</code> .

9. Limiti da conoscere

Niente archivio storico. ComingSoon serve sempre la classifica corrente e ignora le date: il backfill (`backfill_historical_weekly.py`) carica solo le settimane davvero presenti nella pagina. Lo storico si costruisce eseguendo la pipeline ogni settimana.

- Lo schema si modifica solo con nuove migrazioni in `migrations/` ; una migrazione già applicata non va mai modificata (il checksum è verificato).
- Non usare Box Office Mojo come fonte alternativa: il suo `robots.txt` vieta l'accesso automatico.
- Il deposito credenziali non protegge da un programma malevolo eseguito con il tuo stesso account.
- Retention: `backups/` 30 giorni (min. 3 dump), log 90 giorni; le tabelle di audit (`ingestion_runs` , `raw_snapshots` , quarantena) si conservano indefinitamente.